Universidad
Nacional del
Altiplano de Puno

UNIVERSIDAD NACIONAL DEL ALTIPLANO
FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

CAPÍTULO I

Solución de Ejercicios de Programación

ESTRUCTURA DE DATOS – 2026

Docente: Fred Torres-Cruz
Estudiante: Blays Kevin Mamani Quispe
Link de Repositorio GitHub:
https://github.com/blayskevinmamaniquispe-lab/Estructura-de-Datos/tree/main/Ejerccicios

Resumen del Capítulo:

- Ejercicios resueltos en lenguaje C y C++.
- Estructuras secuenciales, condicionales, ciclos y arreglos.
- Modularización mediante funciones y estimación de complejidad algorítmica.

Puno, Perú
2026

Índice

Ejercicio 1. Costo total del procesamiento 2

Ejercicio 2. Registro de una observación 3

Ejercicio 3. Conversión de decimal a entero 4

Ejercicio 4. Operaciones aritméticas 5

Ejercicio 5. Distribución de registros entre nodos 6

Ejercicio 6. Operadores de incremento y asignación 7

Ejercicio 7. Validación de un conjunto de datos 8

Ejercicio 8. Validación de una observación 9

Ejercicio 9. Clasificación de temperatura 10

Ejercicio 10. Media y observaciones superiores a ella (C++) 11

Ejercicio 11. Validación de contraseña (C++) 12

Ejercicio 12. Procesamiento con break y continue (C++) 13

Ejercicio 13. Análisis de edades mediante un arreglo 14

Ejercicio 14. Comparación de complejidad O(n) y O(n2) 15

Ejercicio 15. Funciones para promedio y conteo 16

Conclusión 17

Ejercicio 1. Costo total del procesamiento

Procedimiento: Se leen las horas trabajadas y el costo por hora. El costo total se obtiene multiplicando ambos valores.

CÓDIGO FUENTE EN C: Ejercicio 1

```c
#include <stdio.h>

int main(void) {
    float horas, tarifa, costo;

    printf("Ingrese las horas trabajadas: ");
    scanf("%f", &horas);

    printf("Ingrese el costo por hora: ");
    scanf("%f", &tarifa);

    costo = horas * tarifa;

    printf("\n--- RESUMEN ---\n");
    printf("Horas trabajadas: %.2f\n", horas);
    printf("Tarifa por hora: S/ %.2f\n", tarifa);
    printf("Costo total: S/ %.2f\n", costo);

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio1.c -o ejercicio1
$ ./ejercicio1
Ingrese las horas trabajadas: 42.5
Ingrese el costo por hora: 25.0

--- RESUMEN ---
Horas trabajadas: 42.50
Tarifa por hora: S/ 25.00
Costo total: S/ 1062.50
```

Ejercicio 2. Registro de una observación

Procedimiento: Se utilizan tipos de datos apropiados: entero para el identificador y la edad, float para el promedio, char para la categoría y un entero para representar el estado de validez.

CÓDIGO FUENTE EN C: Ejercicio 2

```c
#include <stdio.h>

int main(void) {
    int identificador, edad, valido;
    float promedio;
    char categoria;

    printf("Identificador: ");
    scanf("%d", &identificador);

    printf("Edad: ");
    scanf("%d", &edad);

    printf("Valor promedio: ");
    scanf("%f", &promedio);

    printf("Categoria (una letra): ");
    scanf(" %c", &categoria);

    printf("Registro valido? (1=Si, 0=No): ");
    scanf("%d", &valido);

    printf("\n--- DATOS DE LA OBSERVACION ---\n");
    printf("Identificador: %d\n", identificador);
    printf("Edad: %d\n", edad);
    printf("Promedio: %.2f\n", promedio);
    printf("Categoria: %c\n", categoria);
    printf("Estado: %s\n", valido == 1 ? "Valido" : "No valido");

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio2.c -o ejercicio2
$ ./ejercicio2
Identificador: 101
Edad: 30
Valor promedio: 56.45
Categoria (una letra): B
Registro valido? (1=Si, 0=No): 1

--- DATOS DE LA OBSERVACION ---
Identificador: 101
Edad: 30
Promedio: 56.45
Categoria: B
Estado: Valido
```

Ejercicio 3. Conversión de decimal a entero

Procedimiento: La conversión explícita mediante `(int)` elimina la parte decimal, sin redondear. La pérdida se calcula restando el entero convertido al valor original.

CÓDIGO FUENTE EN C: Ejercicio 3

```c
#include <stdio.h>

int main(void) {
    float medicion, perdida;
    int valorEntero;

    printf("Ingrese la medicion decimal: ");
    scanf("%f", &medicion);

    valorEntero = (int)medicion;
    perdida = medicion - valorEntero;

    printf("Valor original: %.2f\n", medicion);
    printf("Valor entero: %d\n", valorEntero);
    printf("Parte decimal perdida: %.2f\n", perdida);

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio3.c -o ejercicio3
$ ./ejercicio3
Ingrese la medicion decimal: 18.9

Valor original: 18.90
Valor entero: 18
Parte decimal perdida: 0.90
```

Ejercicio 4. Operaciones aritméticas

Procedimiento: Se leen dos enteros y se calculan suma, diferencia, producto, división entera y residuo. Se comprueba que el segundo número no sea cero antes de dividir.

CÓDIGO FUENTE EN C: Ejercicio 4

```c
#include <stdio.h>

int main(void) {
    int a, b;

    printf("Ingrese el primer entero: ");
    scanf("%d", &a);

    printf("Ingrese el segundo entero: ");
    scanf("%d", &b);

    printf("\nSuma: %d\n", a + b);
    printf("Diferencia: %d\n", a - b);
    printf("Producto: %d\n", a * b);

    if (b != 0) {
        printf("Division entera: %d\n", a / b);
        printf("Residuo: %d\n", a % b);
    } else {
        printf("No se puede dividir entre cero.\n");
    }

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio4.c -o ejercicio4
$ ./ejercicio4
Ingrese el primer entero: 150
Ingrese el segundo entero: 12

Suma: 162
Diferencia: 138
Producto: 1800
Division entera: 12
Residuo: 6
```

Ejercicio 5. Distribución de registros entre nodos

Procedimiento: La división entera determina los registros asignados a cada nodo y el operador módulo determina los registros sobrantes.

CÓDIGO FUENTE EN C: Ejercicio 5

```c
#include <stdio.h>

int main(void) {
    int registros, nodos;

    printf("Numero total de registros: ");
    scanf("%d", &registros);

    printf("Numero de nodos: ");
    scanf("%d", &nodos);

    if (registros >= 0 && nodos > 0) {
        printf("Registros por nodo: %d\n", registros / nodos);
        printf("Registros sin distribuir: %d\n", registros % nodos);
    } else {
        printf("Ingrese registros no negativos y nodos mayores que cero.\n"
            );
    }

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio5.c -o ejercicio5
$ ./ejercicio5
Numero total de registros: 1053
Numero de nodos: 8

Registros por nodo: 131
Registros sin distribuir: 5
```

Ejercicio 6. Operadores de incremento y asignación

Procedimiento: La variable inicia en 4, luego aumenta en 3 y finalmente se multiplica por 2.

CÓDIGO FUENTE EN C: Ejercicio 6

```c
#include <stdio.h>

int main(void) {
    int registros = 4;

    printf("Valor inicial: %d\n", registros);

    registros += 3;
    printf("Despues de incrementar en 3: %d\n", registros);

    registros *= 2;
    printf("Despues de duplicar: %d\n", registros);

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio6.c -o ejercicio6
$ ./ejercicio6
Valor inicial: 4
Despues de incrementar en 3: 7
Despues de duplicar: 14
```

Ejercicio 7. Validación de un conjunto de datos

Procedimiento: El conjunto se acepta únicamente si tiene al menos 100 observaciones válidas y un porcentaje de datos completos igual o superior al 70 %. Ambas condiciones deben cumplirse.

CÓDIGO FUENTE EN C: Ejercicio 7

```c
#include <stdio.h>

int main(void) {
    int observaciones;
    float porcentaje;

    printf("Numero de observaciones validas: ");
    scanf("%d", &observaciones);

    printf("Porcentaje de datos completos: ");
    scanf("%f", &porcentaje);

    if (observaciones >= 100 && porcentaje >= 70.0 && porcentaje <= 100.0)
        {
        printf("El conjunto de datos puede ser analizado.\n");
    } else {
        printf("El conjunto de datos no cumple las condiciones.\n");
    }

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio7.c -o ejercicio7
$ ./ejercicio7
Numero de observaciones validas: 125
Porcentaje de datos completos: 82.5

El conjunto de datos puede ser analizado.
```

Ejercicio 8. Validación de una observación

Procedimiento: Se utiliza el operador lógico AND (`&&`). La observación se incorpora si la edad es como mínimo 18 y el registro está validado.

CÓDIGO FUENTE EN C: Ejercicio 8

```c
#include <stdio.h>

int main(void) {
    int edad, validado;

    printf("Ingrese la edad: ");
    scanf("%d", &edad);

    printf("Registro validado? (1=Si, 0=No): ");
    scanf("%d", &validado);

    if (edad >= 18 && validado == 1) {
        printf("La observacion puede incorporarse.\n");
    } else {
        printf("La observacion no puede incorporarse.\n");
    }

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio8.c -o ejercicio8
$ ./ejercicio8
Ingrese la edad: 22
Registro validado? (1=Si, 0=No): 1

La observacion puede incorporarse.
```

Ejercicio 9. Clasificación de temperatura

Procedimiento: Se utilizan condiciones encadenadas para clasificar la temperatura en tres categorías.

CÓDIGO FUENTE EN C: Ejercicio 9

```c
#include <stdio.h>

int main(void) {
    float temperatura;

    printf("Ingrese la temperatura en grados Celsius: ");
    scanf("%f", &temperatura);

    if (temperatura < 0) {
        printf("Clasificacion: Congelacion\n");
    } else if (temperatura <= 20) {
        printf("Clasificacion: Frio\n");
    } else {
        printf("Clasificacion: Templado\n");
    }

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio9.c -o ejercicio9
$ ./ejercicio9
Ingrese la temperatura en grados Celsius: 14.5
Clasificacion: Frio
```

Ejercicio 10. Media y observaciones superiores a ella (C++)

Procedimiento: Se almacenan diez mediciones en un arreglo y se calcula su suma. La media se obtiene dividiendo la suma entre diez. Después se cuentan los valores mayores que la media.

CÓDIGO FUENTE EN C++: Ejercicio 10

```cpp
#include <iostream>
using namespace std;

int main() {
    double datos[10], suma = 0, media;
    int contador = 0;

    for (int i = 0; i < 10; i++) {
        cout << "Ingrese la medicion " << i + 1 << ": ";
        cin >> datos[i];
        suma += datos[i];
    }

    media = suma / 10;

    for (int i = 0; i < 10; i++) {
        if (datos[i] > media) {
            contador++;
        }
    }

    cout << "Media aritmetica: " << media << endl;
    cout << "Observaciones por encima de la media: " << contador << endl;

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ g++ ejercicio10.cpp -o ejercicio10
$ ./ejercicio10
Ingrese la medicion 1: 12.5
Ingrese la medicion 2: 18.0
Ingrese la medicion 3: 9.2
... [ingreso de mediciones] ...
Media aritmetica: 14.82
Observaciones por encima de la media: 4
```

Ejercicio 11. Validación de contraseña (C++)

Procedimiento: El ciclo `while` solicita la clave hasta que sea correcta. Se incrementa el contador cada vez que se realiza un intento. Para este ejemplo, la clave correcta se establece en 1234.

CÓDIGO FUENTE EN C++: Ejercicio 11

```cpp
#include <iostream>
using namespace std;

int main() {
    const int CLAVE_CORRECTA = 1234;
    int clave, intentos = 0;

    cout << "Ingrese la clave numerica: ";
    cin >> clave;
    intentos++;

    while (clave != CLAVE_CORRECTA) {
        cout << "Clave incorrecta. Intente nuevamente.\n";
        cout << "Ingrese la clave numerica: ";
        cin >> clave;
        intentos++;
    }

    cout << "Acceso permitido.\n";
    cout << "Numero de intentos: " << intentos << endl;

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ g++ ejercicio11.cpp -o ejercicio11
$ ./ejercicio11
Ingrese la clave numerica: 1111
Clave incorrecta. Intente nuevamente.
Ingrese la clave numerica: 1234
Acceso permitido.
Numero de intentos: 2
```

Ejercicio 12. Procesamiento con break y continue (C++)

Procedimiento: Se leen como máximo diez valores. Los negativos se omiten con `continue`; el valor 999 detiene el ciclo con `break`.

CÓDIGO FUENTE EN C++: Ejercicio 12

```cpp
#include <iostream>
using namespace std;

int main() {
    double valor;
    int validos = 0;

    for (int i = 0; i < 10; i++) {
        cout << "Ingrese la medicion " << i + 1 << " (999 para terminar): "
            ;
        cin >> valor;

        if (valor == 999) {
            break;
        }

        if (valor < 0) {
            cout << "Valor negativo: se omite.\n";
            continue;
        }

        validos++;
    }

    cout << "Valores validos procesados: " << validos << endl;

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ g++ ejercicio12.cpp -o ejercicio12
$ ./ejercicio12
Ingrese la medicion 1 (999 para terminar): 15.4
Ingrese la medicion 2 (999 para terminar): -3.2
Valor negativo: se omite.
Ingrese la medicion 3 (999 para terminar): 999
Valores validos procesados: 1
```

Ejercicio 13. Análisis de edades mediante un arreglo

Procedimiento: Se almacenan diez edades. Durante el ingreso se determina la suma, la edad mínima y la máxima. Después se calcula la media y se cuentan las edades superiores a ella.

CÓDIGO FUENTE EN C: Ejercicio 13

```c
#include <stdio.h>

int main(void) {
    int edades[10], suma = 0;
    int minima, maxima, superiores = 0;
    float media;

    for (int i = 0; i < 10; i++) {
        printf("Edad del participante %d: ", i + 1);
        scanf("%d", &edades[i]);

        if (i == 0) {
            minima = maxima = edades[i];
        } else {
            if (edades[i] < minima) minima = edades[i];
            if (edades[i] > maxima) maxima = edades[i];
        }

        suma += edades[i];
    }

    media = (float)suma / 10;

    for (int i = 0; i < 10; i++) {
        if (edades[i] > media) {
            superiores++;
        }
    }

    printf("\nEdad minima: %d\n", minima);
    printf("Edad maxima: %d\n", maxima);
    printf("Media de edades: %.2f\n", media);
    printf("Participantes por encima de la media: %d\n", superiores);

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio13.c -o ejercicio13
$ ./ejercicio13
Edad minima: 19
Edad maxima: 45
Media de edades: 28.40
Participantes por encima de la media: 4
```

Ejercicio 14. Comparación de complejidad $O(n)$ y $O(n^2)$

Procedimiento: El primer procedimiento ejecuta una operación por cada elemento, por lo que realiza n operaciones. El segundo ejecuta dos ciclos anidados de n iteraciones cada uno, realizando n^2 operaciones.

CÓDIGO FUENTE EN C: Ejercicio 14

```c
#include <stdio.h>

int main(void) {
    int n;
    long long operacionesLineales = 0;
    long long operacionesCuadraticas = 0;

    printf("Ingrese el numero de elementos: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("n debe ser un entero positivo.\n");
        return 1;
    }

    /* Procedimiento 1: un recorrido */
    for (int i = 0; i < n; i++) {
        operacionesLineales++;
    }

    /* Procedimiento 2: dos ciclos anidados */
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            operacionesCuadraticas++;
        }
    }

    printf("Operaciones del primer procedimiento: %lld\n",
        operacionesLineales);
    printf("Complejidad aproximada: O(n)\n");

    printf("Operaciones del segundo procedimiento: %lld\n",
        operacionesCuadraticas);
    printf("Complejidad aproximada: O(n^2)\n");

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio14.c -o ejercicio14
$ ./ejercicio14
Ingrese el numero de elementos: 500
Operaciones del primer procedimiento: 500
Complejidad aproximada: O(n)
Operaciones del segundo procedimiento: 250000
Complejidad aproximada: O(n^2)
```

Ejercicio 15. Funciones para promedio y conteo

Procedimiento: La función `calcular_promedio` recibe tres mediciones y retorna su media. La función `contar_superiores` recibe las mediciones y el promedio, retornando cuántas superan dicho promedio.

CÓDIGO FUENTE EN C: Ejercicio 15

```c
#include <stdio.h>

float calcular_promedio(float a, float b, float c) {
    return (a + b + c) / 3.0f;
}

int contar_superiores(float a, float b, float c, float promedio) {
    int contador = 0;

    if (a > promedio) contador++;
    if (b > promedio) contador++;
    if (c > promedio) contador++;

    return contador;
}

int main(void) {
    float m1, m2, m3, promedio;
    int cantidad;

    printf("Ingrese la primera medicion: ");
    scanf("%f", &m1);

    printf("Ingrese la segunda medicion: ");
    scanf("%f", &m2);

    printf("Ingrese la tercera medicion: ");
    scanf("%f", &m3);

    promedio = calcular_promedio(m1, m2, m3);
    cantidad = contar_superiores(m1, m2, m3, promedio);

    printf("Promedio: %.2f\n", promedio);
    printf("Mediciones por encima del promedio: %d\n", cantidad);

    return 0;
}
```

¿ COMPILACIÓN Y EJECUCIÓN EN CONSOLA

```
$ gcc ejercicio15.c -o ejercicio15
$ ./ejercicio15
Ingrese la primera medicion: 14.2
Ingrese la segunda medicion: 18.6
Ingrese la tercera medicion: 11.0
Promedio: 14.60
Mediciones por encima del promedio: 1
```

Conclusión

Los ejercicios desarrollan conceptos fundamentales de programación: variables y tipos de datos, conversiones, operadores aritméticos y lógicos, estructuras condicionales, ciclos, arreglos, funciones y análisis básico de complejidad algorítmica.

Cada programa puede compilarse de manera independiente en Overleaf o cualquier compilador de C/C++.